

Team: Keyboard Gremlins

COS301 Capstone Project
University of Pretoria

Hackathon Platform



Testing Policy

Name	Student Number
Ayush Sanjith	23535424
Jasveer Ramdhaw	23537036
Rene Reddy	23558572
Heinrich Römer	23538181
Kwaku Asiedu (Sam)	18100156

Purpose

This document defines how testing is planned, executed, reviewed, and reported for the Hackathon Platform project. It ensures every contributor follows the same standard for determining when code is sufficiently tested before being merged into the main branch. It also provides transparency into how software quality is enforced throughout development.

Testing Objectives

- Catch regressions before they reach the dev or main branches through mandatory CI checks.
- Verify role based access control is enforced by the backend.
- Validate complete user journeys using end-to-end browser testing.
- Establish a performance baseline under concurrent user load.
- Maintain high code quality through automated analysis and coverage reporting.

Testing Types, Tools and Environments

Test type	Tool	Coverage
Backend Unit tests	JUnit 5 + Mockito	Service layer logic with mocked repositories
Integration tests	JUnit 5 + MockMvc	Controller, service, database and Spring Security config
Frontend Unit Tests	Karma + Jasmine	Angular Components and services
Ent to end tests	Playwright	Complete user workflows using the Chrome browser.
Load Tests	k6	API performance under concurrent load.
Static Code Analysis	SonarCloud	Bugs, vulnerabilities, security, maintainability, reliability and code smells.
Code Coverage	CodeCov	Line and Branch coverage
Linting and formatting	Spotless and ESLint	Code style and formatting.
Frontend performance and Accessibility	Lighthouse CI	Performance, accessibility, best practises and SEO

Load Testing

The k6 load testing suite simulates realistic authenticated traffic.

Test Scenario:

- Ramp from 20 virtual users for 30 seconds
- Increase to 50 virtual users for 1 minute
- Increase to 100 virtual users for 2 minutes
- Ramp down to 0 users over 30 seconds

Each virtual user performs:

1. Login via POST /api/auth/login
2. Retrieve the JWT access token
3. Access GET /api/hackathon using the authenticated token

Defect Management

Automated failures prevent code from being merged until resolved.

Each manually discovered defect should include:

- Steps to reproduce
- Expected behaviour
- Actual behaviour
- Environment
- Severity

Severity Levels:

- Blocker: Prevents core functionality or creates a security issue. Must be fixed before merging.
- Major: Feature behaves incorrectly but the application is usable.
- Minor: Cosmetic or not big issues.

Acceptance Criteria

Before merging into main or dev:

- All CI checks pass
- No new Critical issues from by SonarCloud
- Code coverage does not decrease
- New functionality includes Playwright tests

Before a release or demonstration:

6

- Complete Playwright suite passes
- k6 load test passes all thresholds